

DESIGN AND IMPLEMENTATION OF ORDERS AND ORDER_DETAILS TABLES

-ABINAYA.R(E25AI102)

1. Introduction

The Orders and Order_Details tables are designed to maintain information about customer orders and the products included in every order.

The Orders table stores the main details of an order, such as Customer ID, Order Date, Total Amount, and Order Status.

The Order_Details table stores the individual products included in each order along with the quantity purchased.

2. Orders Entity

The Orders entity contains the overall information related to customer orders.

Attribute	Description
Order_ID	Unique order identifier
Customer_ID	Customer reference
Order_Date	Date on which the order was placed
Total_Amount	Total bill amount
Order_Status	Pending / Shipped / Delivered

3. Order_Details Entity

The Order_Details entity contains the individual products included in an order.

Attribute	Description
Order_Detail_ID	Unique detail identifier
Order_ID	Order reference
Product_ID	Product reference
Quantity	Number of products purchased

4. Creating the ORDERS Table and ORDER_DETAILS Table

The ORDERS table and ORDER_DETAILS is created to store the main information of customer orders.

```
SQL> CREATE TABLE Orders(Order_ID VARCHAR2(7), Customer_ID VARCHAR2(7), Order_Date DATE, Total_Amount NUMBER(10,2), Order_Status VARCHAR2(20));
Table created.

SQL> CREATE TABLE Order_Details(Order_Detail_ID VARCHAR2(7), Order_ID VARCHAR2(7), Product_ID VARCHAR2(7), Quantity NUMBER, Price NUMBER(10,2));
Table created.
```

5. Establishing Primary Keys

A Primary Key is used to uniquely identify each record in a table.

The Order_ID is the Primary Key of the ORDERS table.

The Order_Detail_ID is the Primary Key of the ORDER_DETAILS table.

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT PK_ORDERS PRIMARY KEY(Order_ID);  
Table altered.  
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT PK_ORDER_DETAILS PRIMARY KEY(Order_Detail_ID);  
Table altered.
```

6. Establishing Relationships

The relationship between Customer and Orders is:

Customer (1) → Orders (Many)

One customer can place multiple orders, while each order belongs to one customer.

The relationship between Orders and Products is:

Orders (Many) → Products (Many)

This many-to-many relationship is implemented using the ORDER_DETAILS table.

7. Applying Foreign Key Constraint

The Order_ID in the ORDER_DETAILS table is connected to the Order_ID in the ORDERS table using a Foreign Key.

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_ORDER_DETAILS_ORDER FOREIGN KEY(Order_ID) REFERENCES ORDERS(Order_ID);  
Table altered.
```

8. Applying NOT NULL Constraint

The Order_ID and Product_ID columns in the ORDER_DETAILS table are made mandatory by applying the NOT NULL constraint.

```
SQL> ALTER TABLE ORDER_DETAILS MODIFY(Order_ID NOT NULL, Product_ID NOT NULL);  
Table altered.
```

9. Applying CHECK Constraints

A CHECK constraint is used to make sure that only valid values are entered into the table.

Total Amount Constraint

This constraint ensures that the total amount cannot be negative.

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT CHK_ORDER_STATUS CHECK(Order_Status IN('Pending','Shipped','Delivered'));  
Table altered.
```

Order Status Constraint

```
ALTER TABLE ORDERS ADD CONSTRAINT CHK_ORDER_STATUS CHECK(Order_Status  
IN('Pending','Shipped','Delivered'));
```

This permits only Pending, Shipped, or Delivered as valid order statuses.

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT CHK_QUANTITY CHECK(Quantity>0);  
Table altered.
```

10. Inserting Values into ORDERS

The following records are inserted into the ORDERS table.

```
SQL> INSERT INTO ORDERS VALUES('O101','C101',TO_DATE('20-09-2026','DD-MM-YYYY'),850,'Pending');
1 row created.

SQL> INSERT INTO ORDERS VALUES('O102','C102',TO_DATE('21-09-2026','DD-MM-YYYY'),1200,'Shipped');
1 row created.

SQL> INSERT INTO ORDERS VALUES('O103','C103',TO_DATE('22-09-2026','DD-MM-YYYY'),650,'Delivered');
1 row created.
```

11. Inserting Values into ORDER_DETAILS

The following records are inserted into the ORDER_DETAILS table

```
SQL> INSERT INTO ORDER_DETAILS VALUES('OD101','O101','P101',2,425);
1 row created.

SQL> INSERT INTO ORDER_DETAILS VALUES('OD102','O102','P102',3,400);
1 row created.

SQL> INSERT INTO ORDER_DETAILS VALUES('OD103','O103','P103',2,325);
1 row created.
```

12. Customer Order History

This report displays the customer name, order ID, order date, total amount, and order status for each customer order.

```
SELECT C.CUSTOMER_NAME, O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT, O.ORDER_STATUS AS STATUS
FROM ORDERS O JOIN CUSTOMER C ON O.CUSTOMER_ID = C.CUSTOMER_ID;
```

```
SQL> SELECT C.CUSTOMER_NAME, O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT, O.ORDER_STATUS AS STATUS FROM ORDERS O JOIN CUSTOMER C ON O.CUSTOMER_ID = C.CUSTOMER_ID;
```

CUSTOMER_NAME	ORDER_I	ORDER_DAT	TOTAL_AMOUNT
Devi Pending	0101	01-SEP-26	1500
Radhi Shipped	0102	02-SEP-26	2500
Ajay Delivered	0103	03-SEP-26	1800

13. Product-Wise Order Report

This query finds the number of times each product was ordered and the total quantity sold.

The query uses GROUP BY to organize the order details according to each product.

It calculates:

- No_Of_Times_Ordered – Number of times the product was ordered.
- Total_Quantity_Sold – Total quantity of the product sold.

```
SQL> SELECT Product_ID, COUNT(*) AS No_Of_Times_Ordered, SUM(Quantity) AS Total_Quantity_Sold FROM Order_Details GROUP BY Product_ID;
```

PRODUCT	NO_OF_TIMES_ORDERED	TOTAL_QUANTITY_SOLD
P109	1	5
P105	1	5
P107	1	3
P102	1	3
P110	1	2
P103	1	2
P106	1	2
P104	1	4
P101	1	2
P108	1	2

14. Customer Purchase Analysis

This report analyses customer purchases by identifying customers with the maximum number of orders, the total spending by each customer, and the average order value.

```
SQL> SELECT c.CUSTOMER_NAME, COUNT(*) AS TOTAL_ORDERS, SUM(o.TOTAL_AMOUNT) AS TOTAL_SPENDING, AVG(o.TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE FROM CUSTOMER c JOIN ORDERS o ON c.CUSTOMER_ID = o.CUSTOMER_ID GROUP BY c.CUSTOMER_NAME;
```

CUSTOMER_NAME	TOTAL_ORDERS	TOTAL_SPENDING	AVERAGE_ORDER_VALUE
Anu	1	1200	1200
Ajay	1	1800	1800
Abi	1	2750	2750
Devi	2	5000	2500
Radhi	2	4400	2200
Vijay	1	3200	3200
Sindhu	1	4500	4500
Sanjay	1	2100	2100

```
SQL> SELECT c.CUSTOMER_NAME, COUNT(*) AS TOTAL_ORDERS FROM CUSTOMER c JOIN ORDERS o ON c.CUSTOMER_ID=o.CUSTOMER_ID GROUP BY c.CUSTOMER_NAME HAVING COUNT(*)=(SELECT MAX(ORDER_COUNT) FROM (SELECT COUNT(*) AS ORDER_COUNT FROM ORDERS GROUP BY CUSTOMER_ID));
```

CUSTOMER_NAME	TOTAL_ORDERS
Devi	2
Radhi	2

15. Result

The SQL queries generate useful reports, including:

- Customer order history report
- Product-wise order report
- Customer purchase analysis

16. Conclusion

The Orders and Order_Details tables were successfully designed and implemented using SQL.

The SQL queries provide useful information about customer order history, products ordered, quantities sold, and customer purchase analysis.

Observation

The implemented constraints help maintain valid and consistent order data, while the SQL reports make it easier to understand customer purchases and product-wise order information.